

Simulazione del comportamento di stormi

Chiara Gandolfi, Giorgia Goldoni

7 febbraio 2026

Modifiche intercorse dalla consegna precedente

Il cambiamento più significativo risiede nel superamento del singolo stormo a favore di un sistema multi-flock. Mentre la versione precedente gestiva un'unica popolazione uniforme, il progetto attuale orchestra un `std::vector<Flock>`, permettendo la coesistenza di diverse specie con parametri di interazione indipendenti. Questa evoluzione ha richiesto lo spostamento della logica di coordinamento verso la funzione libera `update(std::vector<Flock>&)`, la quale agisce come un supervisore esterno senza violare l'incapsulamento delle singole classi, consentendo agli stormi di attraversarsi o interagire spazialmente pur mantenendo la propria integrità biologica.

A livello di singolo agente, la struttura `Boid` è stata riprogettata per garantire una gestione della memoria più sicura e coerente. L'introduzione di `std::vector<Boid const*> neighbors` rappresenta un salto di qualità nella gestione della const-correctness: l'algoritmo può ora navigare tra i vicini con la certezza di non poterne alterare accidentalmente lo stato durante la fase di sola lettura. Inoltre, si è scelto di rimuovere il sovraccarico dell'`operator==` per i boid, preferendo legare l'identità dell'agente esclusivamente al suo indirizzo di memoria. Questo evita ambiguità semantiche tipiche di quando si confonde l'uguaglianza dei dati (posizione e velocità) con l'effettiva identità dell'oggetto fisico.

Un'importante revisione ha interessato i limiti spaziali e di velocità, ora gestiti tramite metodi statici per una maggiore pulizia del design. La funzione `speed_limit()` non si limita più a troncare la velocità massima, ma impone anche un limite minimo `min_speed_`. Questo accorgimento impedisce allo stormo di congelarsi in uno stato statico, mantenendo la simulazione sempre dinamica e naturale. Parallelamente, la vecchia logica toroidale è stata sostituita dal metodo `bounce()`, che implementa un rimbalzo elastico contro le pareti. Questa scelta non solo confina visivamente lo stormo, ma aggiunge realismo forzando i boid a reagire ai limiti fisici dell'ambiente di volo.

Il cuore della simulazione risiede nel metodo `update()`. Nella versione precedente, le velocità venivano aggiornate in sequenza nello stesso ciclo, creando un bias asimmetrico per cui l'ultimo boid dello stormo reagiva a velocità già aggiornate, mentre il primo si basava su stati passati. Il progetto attuale risolve il problema tramite un sistema di double buffering: le nuove traiettorie vengono calcolate globalmente e memorizzate nel vettore temporaneo `new_v`, per essere applicate solo al termine del ciclo. Questo garantisce che ogni intervallo temporale sia una fotografia perfettamente simmetrica dello stato del sistema all'istante t .

Infine, l'analisi quantitativa è passata da una semplice statistica descrittiva a un monitoraggio rigoroso tramite l'Errore Standard della Media. Le funzioni `sem_distance()` e `sem_speed()` forniscono ora una misura della precisione della media campionaria, fondamentale per identificare il raggiungimento di un equilibrio dinamico nello stormo. Coerentemente con il resto del progetto, queste funzioni integrano ora un controllo rigoroso tramite eccezioni: il programma impedisce il calcolo di statistiche prive di senso matematico, garantendo che i dati mostrati nel nuovo pannello laterale della GUI siano sempre affidabili e scientificamente fondati.

Progettazione e implementazione

L'architettura del software riflette la modularità richiesta da una simulazione dinamica complessa. Il cuore del sistema risiede nell'interazione tra boid, i quali determinano il proprio movimento attraverso le tre regole fondamentali di separazione, allineamento e coesione. Per garantire l'isolamento delle entità del progetto e facilitare la gestione delle dipendenze, il progetto è stato strutturato all'interno del namespace `pf`.

La gestione spaziale e cinetica si appoggia interamente sulla struttura `Vector`. Per favorire la manutenibilità del codice, tutti gli operatori binari sono stati sviluppati in funzione degli operatori di assegnazione composta. Questo garantisce un unico punto di manutenzione per la logica

algebrica, facilitando future estensioni (es: passaggio al 3D). L'uguaglianza tra vettori è stata definita in base al contenuto semantico delle coordinate e non alla posizione in memoria, garantendo che due vettori siano considerati uguali se descrivono lo stesso spostamento spaziale. La funzione `normalize()` rispetta la definizione matematica, restituendo un vettore nullo se l'input è nullo per prevenire comportamenti indefiniti.

La struttura `Boid` definisce l'agente singolo. La gestione delle interazioni è stata raffinata introducendo un `std::vector<Boid const*>`. L'uso di puntatori a costanti assicura che un boid possa leggere lo stato dei vicini senza il rischio di modifiche accidentali. A differenza di `Vector`, per `Boid` si è scelto di non sovraccaricare l'operatore di uguaglianza: l'identità dell'agente è definita univocamente dal suo indirizzo di memoria (`&b1 == &b2`) e non dal suo contenuto.

La classe `Flock` implementa la simulazione collettiva, proteggendo i dati tramite l'incapsulamento privato. Le tre regole di volo sono implementate come metodi privati e vengono applicate tenendo conto dei boid vicini.

La velocità di separazione `v1_()` evita che i boid collidano tra loro. Usando la distanza come fattore per determinare la nuova velocità permette di avere un'accelerazione graduale del boid che si allontana. In particolare, è stata implementata con un ciclo esplicito per massimizzare la leggibilità e la semplicità, evitando le complessità di algoritmi generici che si adattano male al filtraggio di elementi basato sulla distanza di sicurezza `ds_`.

La velocità di allineamento `v2_()` permette ai boid di procedere nella stessa direzione dello stormo, mentre la velocità di coesione `v3_()` induce i boid a volare verso il centro di massa dei vicini. Le ultime due regole di volo sfruttano l'algoritmo `std::accumulate` per una semantica più chiara. Questa scelta permette l'inizializzazione diretta delle somme vettoriali e garantisce l'immutabilità dei dati intermedi. L'uso di `static_cast<>` previene conversioni implicite durante il calcolo del centro di massa e delle medie cinetiche.

L'aggiornamento dei vicini in `update_neighbours()` avviene svuotando preventivamente il contenitore dei puntatori e assicurandosi che la vicinanza sia sempre calcolata in base alla distanza euclidea corrente rispetto al parametro `d_`.

I comportamenti limite dello stormo sono gestiti dai metodi statici `speed_limit()` e `bounce()`. La scelta della parola chiave `static` deriva dal fatto che queste funzioni operano esclusivamente su un riferimento a un `Boid` e non necessitano dello stato specifico di un'istanza di `Flock`. Ciò permette di accedere a parametri costanti senza dover ricorrere a getter pubblici. In particolare `speed_limit()` limita la velocità della simulazione nel range `[min_speed_, max_speed_]`. Mentre `bounce()` simula il comportamento di un urto elastico contro pareti rigide, mantenendo lo stormo confinato nell'area di gioco.

Il metodo `update()` orchestra l'evoluzione temporale dello stormo. Per prevenire il bias asimmetrico che si creerebbe se un boid reagisse alla nuova velocità di un vicino già aggiornato nello stesso frame, il sistema calcola le nuove velocità per tutti i componenti memorizzandole in un vettore temporaneo `new_v`. Solo una volta completato il calcolo per l'intero stormo, le velocità vengono applicate, i limiti imposti e le posizioni aggiornate. Questo garantisce che ogni agente si muova basandosi esclusivamente sulla "fotografia" dello stato globale all'istante `t`.

Il coordinamento superiore è affidato alla funzione libera `update(std::vector<Flock>&)`, la quale permette di gestire più stormi estendendo le funzionalità della classe senza violarne l'incapsulamento.

La generazione iniziale è affidata a `generate_random_boids()`, che utilizza l'header file `<random>` per una distribuzione uniforme, garantendo tramite eccezioni che la popolazione `N` sia valida.

Le funzioni libere `mean_distance()`, `sem_distance()`, `mean_speed()`, `sem_speed()` analizzano quantitativamente l'evoluzione del sistema, calcolando medie ed Errori Standard della Media (SEM). Queste funzioni integrano controlli di robustezza: se la popolazione non è sufficiente a garantire una base statistica, viene sollevata un'eccezione `std::runtime_error` per prevenire calcoli matematicamente inconsistenti.

Infine, la visualizzazione grafica tramite SFML separa nettamente la resa estetica dal monitoraggio dei dati. Attraverso l'uso strategico di diverse `sf::View`, la finestra è suddivisa in un'area azzurra per la simulazione e un pannello nero per le statistiche. Quest'ultimo supporta lo scrolling manuale e l'aggiornamento in tempo reale dei dati numerici di ogni stormo. I boid sono rappresentati tramite sprite animati, orientati automaticamente nella direzione del vettore velocità, e offrono una percezione fluida del comportamento collettivo a 60 FPS.

Librerie esterne da installare e strumenti di sviluppo

Per garantire la corretta compilazione ed esecuzione del progetto è richiesta la libreria grafica SFML nella versione 2.6 o superiore, la cui installazione deve essere effettuata tramite il comando:

```
sudo apt install libsFML-dev
```

Per quanto riguarda il rispetto degli standard qualitativi e di correttezza richiesti, il progetto si avvale strumenti specifici come Doctest e clang-format. Doctest è un framework di unit testing che non richiede installazioni di sistema poiché è gestito come libreria header-only tramite il file `doctest.h` incluso nella cartella del progetto e integrato nel processo di build. Al contrario, clang-format non è una libreria da compilare ma uno strumento di formattazione che assicura uniformità e coerenza del codice tra diversi sviluppatori. Il suo utilizzo è vincolato al file di configurazione `.clang-format` consegnato con l'elaborato.

Esecuzione del programma

Il build del progetto è gestito interamente tramite CMake, sistema configurato per operare con il generatore Ninja Multi-Config. La procedura standard prevede l'inizializzazione dell'ambiente attraverso il comando:

```
cmake -S . -B build -G"Ninja Multi-Config"
```

Questa operazione permette la successiva compilazione ed esecuzione del software nelle diverse configurazioni previste. In particolare, per la modalità di Debug si utilizzano rispettivamente i comandi:

```
cmake --build build --config Debug  
./build/Debug/boids
```

Mentre per la modalità di Release i comandi di riferimento sono:

```
cmake --build build --config Release  
./build/Release/boids
```

Durante l'esecuzione, potrebbe apparire sul terminale il seguente messaggio:

```
Setting vertical sync not supported
```

Si specifica che tale segnalazione è generata internamente e causata da una limitazione dei driver video interfacciati con SFML nel gestire la sincronizzazione verticale della finestra grafica. La diagnostica di parti terze non pregiudica in alcun modo la correttezza o la stabilità del programma poiché il limite di frame rate è gestito via software tramite la funzione `setFramerateLimit()`, la quale assicura una velocità di esecuzione costante e indipendente dal supporto hardware al vertical sync.

Parametri di input e formato di output

La fase di input è stata ridotta al minimo essenziale per la configurazione dell'ambiente. Il programma interagisce con l'utente tramite terminale, richiedendo il numero di stormi desiderati (n) e i parametri specifici per ogni gruppo: popolazione (N), distanze caratteristiche (d , ds) e coefficienti di Reynolds (s , a , c). Per osservare un comportamento collettivo armonioso si consiglia l'utilizzo di stormi di qualche decina di boids con i seguenti parametri di riferimento:

```
--- Multi-Flock Simulation Setup ---  
Enter number of flocks (n): 3  
  
Flock #1  
Enter number of boids (N): 75  
Enter interaction distance (d): 40  
Enter separation distance (ds): 28  
Enter separation parameter (s): 0.07  
Enter alignment parameter (a): 0.07  
Enter cohesion parameter (c): 0.09
```

Il programma esegue una validazione rigorosa e immediata dei parametri di input: ogni valore inserito viene controllato sia sotto il profilo del tipo (verificando lo stato dello stream `!std::cin`), sia per quanto riguarda il dominio di validità (direttamente nel costruttore della classe `Flock`). Sono accettati valori strettamente positivi per le distanze (con $ds > d$), mentre i parametri di volo devono risiedere nell'intervallo $(0, 1)$. Se si inseriscono input non validi, il software solleva un'eccezione, notifica l'utente con un adeguato messaggio d'errore e termina l'esecuzione.

Una volta avviata la simulazione, l'interfaccia grafica si articola in due regioni funzionali distinte. La porzione principale è caratterizzata da uno sfondo azzurro e ospita la resa dinamica della simulazione: qui i boids vengono visualizzati con colorazioni differenti in base allo stormo di appartenenza ed è possibile osservarne le mutue interazioni spaziali. A questa finestra grafica si affianca un pannello laterale a sfondo nero dedicato alle statistiche. Nello specifico, ogni secondo vengono aggiornate distanza media e velocità media dei boids di ogni stormo. Entrambe le grandezze sono integrate dal calcolo dell'Errore Standard della Media (SEM), fornendo così un'indicazione rigorosa sulla coesione interna e sulla stabilità cinetica del gruppo.

La simulazione viene interrotta chiudendo la finestra grafica oppure premendo `Esc` sulla tastiera.

Interpretazione dei risultati ottenuti

L'osservazione della simulazione permette di confermare la validità del modello di Reynolds nel descrivere l'emergere di comportamenti collettivi complessi.

L'applicazione bilanciata delle tre regole sociali produce dinamiche visibilmente differenziate: un valore di allineamento significativo induce un movimento fluido e coordinato dei boid, mentre un parametro adeguato di coesione favorisce la rapida convergenza dei boid verso il centro di massa, portando alla formazione di cluster stabili. Sotto il profilo quantitativo, questo fenomeno è reso evidente dall'andamento dell'Errore Standard della Media della velocità: dopo i primi istanti di simulazione, tale valore tende a ridursi drasticamente e a stabilizzarsi verso il minimo, a testimonianza del fatto che gli agenti hanno uniformato la propria direzione di volo.

L'implementazione multi-flock arricchisce ulteriormente lo scenario, mostrando come popolazioni caratterizzate da parametri differenti possano attraversarsi o sovrapporsi senza influenzarsi a vicenda in base ai rispettivi raggi d'azione. Questo simula correttamente l'indipendenza biologica tra diverse specie che condividono lo stesso spazio aereo, permettendo di osservare come ogni stormo mantenga la propria coesione interna e la propria stabilità statistica indipendentemente dalla presenza di agenti esterni.

L'attuale implementazione di una velocità minima garantisce che lo stormo non collassi in uno stato statico, ma mantenga un movimento di crociera perpetuo che simula fedelmente la biologia dei volatili.

Al raggiungimento dei confini della finestra grafica, l'integrità della simulazione è garantita dalla funzione `bounce()`, che introduce un comportamento di rimbalzo elastico. Quando un boid raggiunge il perimetro dell'area di volo, la sua velocità viene invertita rispetto alla normale della parete; questo non solo mantiene l'integrità del sistema confinando i boids nello spazio visibile, ma forza lo stormo a riorganizzarsi continuamente e a reagire collettivamente agli urti contro i limiti fisici della simulazione.

Tuttavia, l'efficacia del modello incontra limiti naturali legati alla densità della popolazione e alla complessità computazionale. Poiché il calcolo dei vicini segue una logica $O(N^2)$, con stormi superiori ai 150 agenti si avverte un calo della fluidità dovuto all'elevato numero di confronti spaziali. In queste condizioni, il volo può apparire più disturbato e caratterizzato da bruschi cambiamenti di direzione. Questo fenomeno è causato dalla frequenza con cui i boids tentano di correggere la rotta per rispettare il raggio di separazione; poiché la rotazione dello sprite è legata istantaneamente al vettore velocità, ogni brusca correzione per evitare una collisione si traduce in un rapido cambio di orientamento tra i frame.

Infine, lo stormo tende infine a compattarsi e a stabilizzare la propria velocità, evolvendo verso uno stato di quiete dinamica altamente coordinata.

Strategia di test

La correttezza del programma è stata verificata da una suite di unit test implementata tramite il framework `Doctest` e integrata nel sistema di build `CMake`. Per garantire l'affidabilità del software in ogni scenario, i test sono stati eseguiti sistematicamente sia in configurazione `Debug` che `Release` attraverso i comandi:

```
cmake --build build --config Debug --target test
cmake --build build --config Release --target test
```

In primo luogo, è stata verificata l'algebra della struttura `Vector`: sono stati validati tutti gli operatori aritmetici e le funzioni geometriche, ponendo particolare attenzione alla gestione dei casi limite come la divisione per zero e la normalizzazione di vettori nulli.

Successivamente, la suite si concentra sull'invarianza della classe `Flock`. Attraverso la verifica delle eccezioni nel costruttore, è stato garantito che il programma impedisca la creazione di stormi privi di boid o caratterizzati da parametri fisici inconsistenti. Un punto cardine riguarda la relazione tra le distanze: i test confermano il lancio di un'eccezione qualora la soglia di separazione (`ds_`) risulti superiore o uguale al raggio di interazione (`d_`), condizione che renderebbe imprevedibile la logica di volo.

La componente cinematica è stata invece validata testando le funzioni `speed_limit()` e `bounce()`: inserendo valori di posizione e velocità esterni ai range consentiti, si è verificata la capacità del sistema di riportare gli agenti entro i confini dell'area di simulazione e di mantenere il modulo della velocità all'interno del range `[min_speed_, max_speed_]`.

La logica di volo è stata analizzata nel dettaglio testando singolarmente le tre regole di Reynolds `v1_()`, `v2_()` e `v3_()` tramite metodi `getter`. Creando scenari controllati, è stato possibile verificare che `update_neighbours()` identifichi correttamente i vicini e che le componenti della velocità riflettano con precisione i parametri di separazione, allineamento e coesione impostati. Il test sulla funzione `update()` completa questa fase, assicurando che l'evoluzione temporale della posizione e della velocità avvenga senza errori di accumulo.

Infine, la correttezza statistica è stata verificata confrontando i risultati delle funzioni di media ed Errore Standard della Media con calcoli manuali pre-validati. I test hanno confermato che la simulazione gestisce correttamente i casi limite, lanciando eccezioni specifiche quando la popolazione risulta insufficiente.

Uso di sistemi di Intelligenza Artificiale generativa

In conformità con la [policy di Ateneo](#) e le linee guida fornite durante il corso, si dichiara l'utilizzo di sistemi di Intelligenza Artificiale Generativa a supporto dello sviluppo del progetto. L'impiego di tali strumenti è stato limitato a funzioni meramente ausiliarie legate all'implementazione grafica con SFML. Nello specifico, GenIA è stata consultata per suggerimenti su funzioni di posizionamento spaziale degli elementi grafici, aggiornamento delle statistiche, ottimizzazione delle routine di scrolling nelle `sf::View` e gestione della chiusura della finestra grafica. Ogni frammento di codice suggerito è stato sottoposto a revisione critica, verificandone la correttezza semantica e chiedendo approfondimenti per avere una minima capacità di rendere conto dei risultati generati.

Non sono stati utilizzati strumenti di autocompletamento continuo (es. Copilot dentro VSCode) all'interno dell'editor durante lo sviluppo della logica algoritmica dei boids, privilegiando invece la scrittura manuale del codice per garantire la piena comprensione delle strutture logiche.